발표용 연구 Brief — SMT2020 LV/HM 팹 디스패칭 + DSM 기반 의사결정

> 본 문서는 발표 슬라이드 작성을 위한 자료 모음입니다. 각 절은 슬라이드 1~2장 분량으로 가공하기 좋도록, "사실(데이터/숫자)" 과 "발표 메시지(왜 중요한가)" 를 함께 적어두었습니다.

---

0. 한 장 요약 (Executive summary)

- **문제**: 반도체 팹은 수백 개 설비, 수백 step 에 달하는 라우트, 납기·우선순위·setup·CQT·고장·PM 같은 수많은 제약이 동시에 얽혀있는 대규모 동적 스케줄링 문제다. 작업자(또는 dispatcher) 가 매 순간 다음 어떤 lot 을 어떤 tool 에 보낼지 결정해야 하지만, 결정의 단위가 *한 tool* 이라는 점에서 본질적으로 **국소적(greedy)** 이다.
- **데이터**: 공개 테스트베드 **SMT2020 Dataset 2 (LV/HM)**. 10개 제품, 106 toolgroup, 1,313 tool, 12 공정 area, 라우트당 242–583 step, 167,129 개 lot release record (variable due dates), full PM/breakdown/CQT/setup/rework/sampling/batch/cascading 데이터.
- **시뮬레이터**: SMT2020 데이터를 그대로 읽어 동작하는 자체 구현 이산사건 시뮬레이터 (Python). dispatch policy 교체 가능, capacity·PM·고장·setup·이송 override 가능, lot/tool/toolgroup/event 단위 metric 산출.
- **베이스라인**: 현장 실무 규칙을 단순화한 7개 dispatch rule (FIFO, SPT, EDD, Critical Ratio, Priority+CR+FIFO ...). RL agent 도 베이스라인의 한 종류로 간주. 모두 **tool-level greedy** 라는 동일한 한계를 공유한다.
- **우리 접근**: 라우트가 만드는 **공정 간 연결 구조**(Design Structure Matrix, DSM) 를 명시적으로 추출해, "지금 이 lot 을 보내면 강하게 연결된 후속 공정의 부하/지연이 어떻게 변하는가" 를 의사결정에 반영. 즉 *tool 한 대* 에 갇힌 greedy 시야를, *팹 전역의 흐름 모듈* 단위로 넓힌다.

---

1. 연구 배경 — 왜 반도체 팹 디스패칭이 어려운가

1.1 팹 운영의 본질적 복잡성

반도체 wafer 는 한 lot 이 fab 에 들어와 출하되기까지 수백 개 공정을 거치며, 같은 area (예: Litho, Dry-etch) 를 *여러 번 반복* 방문한다 (re-entrant flow). 한 공정이 끝나면 다음 공정이 미리 정해져 있긴 하지만, 같은 공정을 처리할 수 있는 tool 이 여러 대(=toolgroup) 이고, 이들 tool 은 다른 lot 을 처리하느라 바쁠 수도, PM 으로 멈춰 있을 수도, 고장 났을 수도 있다.

1.2 LV/HM (Low-Volume / High-Mix) 의 특수성

대량 양산형 (HVLM, High-Volume Low-Mix) fab 과 달리, **LV/HM 팹은 제품 종류가 많고 각 제품의 수량은 적다**. SMT2020 dataset 2 는 정확히 이 LV/HM 시나리오를 모사한다.

- 제품 종류: 10
- 라우트 길이: 242 ~ 583 step (제품별 매우 다름)
- 사용 area: 모든 제품이 12 area 모두 사용
- Hot lot 비율: 약 2.5% (총 lot 의)
- 결과: 같은 toolgroup 도 시점마다 매우 다른 제품/step 의 wafer 를 처리해야 함 → setup, batch 형성, dedication, 학습 효과가 모두 끼어듦

1.3 의사결정의 단위

실제 fab MES (Manufacturing Execution System) 에서 dispatch 는 **"한 tool 이 idle 이 되어 다음 lot 을 골라야 할 때마다"** 일어나는 *국소* 의사결정이다. 이때 dispatcher 는:

- 그 tool 앞에 와 있는 lot 들의 우선순위, 납기, 처리시간, setup 여부 등을 확인하고
- *한 명의 tool* 시점에서 가장 좋은 lot 을 고른다

이 결정은 빠르고 robust 해야 하지만, **이 lot 을 보냄으로써 fab 전체에서 어떤 일이 벌어지는가** (다음 공정 큐가 어떻게 변하는가, 병목이 더 막히는가, hot lot 이 다른 곳에서 어떻게 이동하는가) 는 거의 고려되지 않는다. 본 연구의 출발점은 바로 이 **시야의 좁음**이다.

1.4 메시지

> *"Tool 단위 greedy 의사결정은 빠르지만, fab-level 최적이 아닐 수 있다. 의사결정 시점에 '관련된 다른 공정' 을 함께 보면 더 global 한 결정을 할 수 있지 않을까?"*

---

2. 데이터셋 — SMT2020 Dataset 2 (LV/HM 팹)

2.1 출처와 의의

> Denny Kopp, Michael Hassoun, Adar Kalir, Lars Mönch. "**SMT2020 — A Semiconductor Manufacturing Testbed**," *IEEE Transactions on Semiconductor Manufacturing*, Vol. 33, No. 4, 2020, pp. 522–531. DOI: 10.1109/TSM.2020.3001933

- **공개 testbed**: 학계가 동일한 가정 위에서 알고리즘을 비교할 수 있도록 만들어진 표준 데이터.
- 두 종류 시나리오: **HVLM** (대량/소품종, dataset 1) 과 **LVHM** (소량/다품종, dataset 2). 본 연구는 LV/HM (dataset 2).
- 원본은 Excel 17 sheet. 본 저장소는 이를 CSV 17 개로 변환해 공개.

2.2 데이터 규모 (한눈 표)

항목	값
제품 (Product)	10
라우트 (Route)	10 (제품별 1개)
Toolgroup (= 동일 성격 tool 묶음, work center)	106
총 Tool 수	1,313
공정 Area	12
Lot 당 wafer 수	25
Lot release record (variable due dates)	167,129
Regular lot / Hot lot / Super-hot lot	162,800 / 4,180 / 149
시뮬레이션 horizon	2018-01-01 ~ 2025-12-31
Setup 정의된 step	401
Lot-to-Lens (LTL) dedication step	73

항목	값
Rework 가능 step	52
Metrology sampling step	955
CQT 제약 step	264
Batch step	135
Cascading interval step	1,668

2.3 공정 Area 와 설비 구성

12개 area 와 toolgroup, tool 분포:

Area	Toolgroup 수	Tool 수	비고
`Diffusion`	10	73	Batch 공정 (Same Product/Step, Wafer 단위)
`Dielectric`	10	54	일부 cascading
`Dry_Etch`	21	312	가장 많은 toolgroup, 일부 setup-dependent
`Implant`	9	36	Cascading + setup-avoidance dispatching
`Litho`	11	170	Litho-stepper 와 track 분리, LTL dedication
`Litho_Met`	4	45	Litho metrology
`Def_Met`	7	15	Defect metrology
`TF`	11	79	Thin film, cascading 다수
`TF_Met`	2	4	Thin film metrology
`Planar`	6	26	CMP 공정군, cascading
`Wet_Etch`	14	99	Cascading 다수, batch tool 일부
`Delay_32`	1	400	"Delay" tool, 의도적 대기시간 부여 (재산화 등)

발표 포인트:

- toolgroup 마다 tool 수가 매우 불균등 (1대 ~ 400대). 한 toolgroup 이 막히면 그 area 전체가 막히기도, 한 toolgroup 이 여유로워도 같은 area 의 다른 toolgroup 에 부하가 쏠릴 수도 있다.
- `Delay_32` 는 *공정* 이 아니라 *대기 자체* 가 spec 인 step (400 대 = 사실상 capacity 제약 없음).
- Cascading / Batching 이 area 마다 다르다 → dispatch 결정이 "한 lot" 이 아니라 "여러 lot 을 동시에 묶어서" 처리되는 경우도 있다.

2.4 라우트 — 제품별 공정 시퀀스

- 라우트 = 제품 한 개가 fab 입력에서 출하까지 거치는 공정 step 시퀀스.
- 길이: Product 5 (242 step, 가장 짧음) ~ Product 3 (583 step, 가장 김).
- 모든 제품이 12 area 를 모두 방문하지만, **방문 순서와 횟수, 사용하는 toolgroup 의 분포는 제품마다 다르다**.

제품	Step 수	사용 toolgroup 수
Product 1	521	104
Product 2	529	101
Product 3	583	105
Product 4	343	82
Product 5	242	70
Product 6	293	80
Product 7	353	92
Product 8	375	92
Product 9	384	94
Product 10	390	94

라우트 step 의 주요 컬럼:

컬럼	의미
`STEP`, `STEP DESCRIPTION`	step 순번과 설명 (e.g. "004_Diffusion")
`AREA`, `TOOLGROUP`	처리 area / toolgroup
`PROCESSING UNIT`	`Wafer` / `Lot` / `Batch` (시간 단위 결정)
`PROCESSINGTIME` (분포, 평균, offset)	확률적 처리시간
`BATCH MIN/MAX`	batch 공정의 묶음 크기
`SETUP`, `WHEN`, `SETUP TIME`	setup 발생 조건
`STEP FOR LTL DEDICATION`	Lithography lot-to-lens dedication 제약
`REWORK PROBABILITY`, `STEP FOR REWORK`	재작업 확률과 되돌아갈 step
`SAMPLING %`	Metrology 샘플링
`STEP FOR CQT`, `CQT`, `CQTUNITS`	Critical Queue Time 제약

2.5 Lot release / 납기

두 종류의 release 파일:

1. **`Lotrelease.csv` (요약, 21 row)** — 제품별 반복 release pattern (regular + hot lot). interval, lots-per-release, 기준 due date 가 들어있다.
2. **`Lotrelease - variable due dates.csv` (167,129 row)** — 개별 lot 단위 *expanded* release. 시뮬레이터가 그대로 읽어 lot 을 fab 에 투입하는 데 쓰인다. 각 lot 의 release time, due date, lot type (Regular/Hot/Super-hot), priority 가 들어 있다.

규모 감각:

- 8년 horizon, 167K lot → 총 wafer ≈ 4.18M 장 (lot 당 25 wafer 기준).
- Hot lot ≈ 2.5%, super-hot lot ≈ 0.09%.
- 제품 간 release 양은 거의 균등 (LV/HM 의 정의대로).

2.6 운영 제약 (operational constraints)

```
PM          : 292 이벤트   (calendar / counter-based scheduled downtime)
Breakdown   : 11 이벤트    (area 단위 random failure, exponential TTF/TTR)
Setup       : 13 전환규칙  (sequence-dependent setup time 7~80분)
Transport   : 1 규칙       (모든 tool 간 이동 5~10분, uniform(7.5±2.5))
CQT         : 264 step     ("이 step 끝나고 N분 안에 다음 step 도착" 시간제약)
Rework      : 52 step      (특정 확률로 이전 step 으로 되돌아감)
Sampling    : 955 step     (metrology step 의 일부만 처리)
Batch       : 135 step     (Diffusion 등에서 같은 product/step 묶음)
Cascading   : 1,668 step   (TF, Wet-Etch, Planar 등에서 연속 처리 간격)
LTL         : 73 step      (Litho lot-to-lens dedication, 같은 stepper 강제)
```

이 모든 제약이 **동시에** 작동한다. 즉 dispatcher 는 한 결정에서 여러 제약을 동시에 만족시켜야 한다.

2.7 데이터셋 메시지 (발표 한 줄)

> *"SMT2020 dataset 2 는 학계 공인 LV/HM 반도체 팹의 표준 testbed 다. 12 공정 area, 1,300+ 설비, 170K lot 의 8년 운영을 실제와 가까운 수준의 모든 제약(PM/고장/CQT/Setup/Rework/Batch/Cascading) 과 함께 재현한다."*

---

3. 시뮬레이터 — SMT2020 직접 구현 DES

3.1 목적

상용 DES (AnyLogic, AutoSched AP 등) 에 묶이지 않고, **SMT2020 데이터를 그대로 읽어 동작하는 reproducible 한 Python DES** 를 만든다. 목적:

1. **policy 비교** — dispatch rule, RL agent, DSM-aware policy 등을 같은 fab 위에서 동등 비교.
2. **재현성** — config (JSON) 만 공유하면 누구나 동일 결과 재생산.
3. **유연성** — capacity, PM, breakdown, setup, transport, release scenario 를 코드 수정 없이 override 가능.

3.2 시뮬레이터 아키텍처

```
configs/default_simulation.json  ──┐
                                   │
dataset/*.csv  ──>  io.load_model(config)  ──>  FabModel
                                                  │
                                        Simulator(model, config).run()
                                                  │
                                                  ▼
                                     SimulationResult (lots, events, summary)
```

- `src/simulator/config.py` — `SimulationConfig` (dataclass + JSON loader, override 지원)
- `src/simulator/model.py` — `Route`, `RouteStep`, `ToolGroup`, `Tool`, `Lot` 등 도메인 객체
- `src/simulator/io.py` — CSV 들을 읽어 불변 `FabModel` 구성
- `src/simulator/engine.py` — DES 본체 (event loop, queueing, dispatching, 처리, 다운타임)
- `src/simulator/policies.py` — 14개 dispatch rule registry (FIFO ~ Priority+CR+FIFO)
- `src/simulator/analysis.py` — lot/tool/toolgroup/event 단위 metric 집계
- `src/simulator/cli.py` — CLI entrypoint (`smt2020-sim`)

3.3 모델 객체와 데이터 매핑

시뮬레이터 객체	데이터 출처
`Product`	`Route_Product_*.csv` 의 ROUTE name (1:1)
`Route`, `RouteStep`	`Route_Product_*.csv` 의 행 (각 step)
`ToolGroup`, `Tool`	`Toolgroups.csv` (capacity, batch/cascade flag, dispatch rule)
`Lot`	`Lotrelease - variable due dates.csv` 의 row (개별 release)
처리시간 분포	route step 의 PROCESSINGTIME DISTRIBUTION/MEAN/OFFSET (uniform/exponential 등)
PM	`PM.csv` (calendar-based 또는 counter-based)
Breakdown	`Breakdown.csv` (area 단위 exponential failure)
Setup	`Setups.csv` (sequence-dependent transition)
Transport	`Transport.csv` (모든 fab→fab 이동, uniform)

3.4 시뮬레이션 흐름 (event loop)

1. **Release**: 다음 lot 의 release 시각이 되면 fab 에 투입. lot 은 자기 라우트의 첫 step 으로 진입.
2. **Queueing**: lot 은 해당 step 의 toolgroup 큐에 합류. 큐 진입 시각 (waiting_since) 기록.
3. **Dispatch**: 어떤 tool 이 idle 이 되거나 새 lot 이 큐에 들어왔을 때, 해당 tool 이 처리 가능한 lot 들 중 **dispatch policy** 가 1개를 선택.
   - 선택된 lot 은 transport time 후 tool 에 도착, loading → setup (필요 시) → processing → unloading → 다음 step 으로.
   - Batch / cascading tool 은 여러 lot 이 동시에 entry.
4. **PM / Breakdown**: tool/area 단위로 capacity 가 일시적으로 줄거나 0 이 됨. 진행 중인 처리는 영향 없으나, 신규 dispatch 막힘.
5. **CQT 추적**: cqt-start step 에서 시작해 cqt-end step 도착 사이의 경과 시간을 기록, 제한을 넘기면 violation 으로 마킹.
6. **Rework**: 확률적으로 이전 step 으로 되돌림.
7. **Sampling**: 일부 lot 만 처리 (skip 가능).
8. **Completion**: 라우트 마지막 step 끝나면 lot 완성, cycle time / tardiness / on-time 여부 기록.

이 과정은 모두 stochastic (process time, transport, breakdown). seed 고정 시 완전 재현 가능.

3.5 운영 제약 처리 요약

- **Cascading**: tool 의 `CASCADINGTOOL` flag + step 의 `CASCADING INTERVAL` 이 모두 있을 때만 적용. 후속 wafer 가 정해진 간격으로 연속 processing 됨.
- **Batching**: 현재 데이터의 `BATCHCRITERION = "Same Product and Same Step"`. 같은 제품/같은 step lot 만 묶고, batching unit 이 `Wafer` 면 wafer 수 기준으로 batch 크기 계산.
- **Setup**: route step 의 `SETUP` 상태가 바뀔 때 `Setups.csv` 의 sequence-dependent setup time 적용.
- **PM/Breakdown**: tool group capacity 를 시간에 따라 줄였다가 복구.
- **CQT**: violation 카운트, 최대 lateness, 검사 횟수, violation rate 까지 기록.

3.6 결과 지표 (metric)

매 시뮬레이션 종료 후 다음 metric 이 자동 산출됨:

- **Throughput**: completed lot 수, completion ratio
- **Cycle time**: lot 별 평균 / 분포
- **Tardiness**: 평균 / max / tardy lot 수 / on-time ratio
- **CQT**: violation 횟수, 평균 lateness, max lateness, violation rate
- **Tool / Toolgroup utilization**: 처리 시간, 대기 시간, downtime
- **Snapshot at simulation end**: 미완성 lot 의 현 step / 진척도

이 metric 들이 정책 비교의 ground truth 역할을 한다.

3.7 발표 메시지

> *"공인 데이터셋 + 자체 구현 DES 로 모든 정책을 동일한 fab 위에서 동등 비교한다. Hyperparameter (설비 수, PM 강도, 이송 속도, 등) 는 JSON config 에서 자유롭게 override 가능."*

---

4. 도메인 지식 — 실제 작업자가 wafer 를 선택하는 규칙

이 절은 슬라이드에서 가장 청중에게 친숙한 부분이 될 것이다. *왜 baseline rule 들이 이런 형태인가* 를 이해하면, 우리가 *왜 그것을 넘어서려 하는가* 도 자연스럽게 따라온다.

4.1 디스패치 결정의 기본 단위

현장 dispatcher 의 결정은 다음 형태로 일어난다:

> **"Toolgroup X 의 tool i 가 idle 이 되었다. X 큐에 lot {A, B, C, ...} 가 대기 중이다. 무엇을 다음으로 보낼까?"**

이 결정은:

- *그 tool 한 대* 의 시점에서 일어난다 → **국소적**
- 한 번에 한 lot 을 고른다 (batch tool 은 여러 lot)
- 어느 lot 이 fab 전체에서 어떤 영향을 줄지는 보지 않는다

작업자가 머릿속으로 굴리는 1차 규칙들은 거의 모두 이 단위에서 정의된다.

4.2 우선순위 (Hot Lot 처리)

- **Hot lot**: 전략적으로 빠르게 빼야 하는 lot (개발 lot, 고객 deadline 임박 lot 등). priority 20.
- **Super-hot lot**: 무조건 최우선. priority 30.

- 도메인 규칙: `SuperHotLotFIRST_and_Reservation` — super-hot lot 은 즉시 우선, 일반 hot lot 은 reservation (병목 설비를 미리 잡아둠) 까지 적용.

발표 메시지: *"우선순위는 매우 강한 신호. 다른 어떤 규칙보다 먼저 적용된다."*

4.3 Critical Ratio (CR) — 납기 기반

$$ \text{CR} = \frac{\text{remaining time to due}}{\text{remaining nominal processing time}} $$

- CR < 1: 이미 늦었거나 거의 늦었다. 무조건 빨리 빼야 한다.
- CR ≈ 1: 정상.
- CR > 1: 여유 있음.
- 작업자는 lot 을 처리할 때 직관적으로 "이거 due 가 임박했나?", "남은 공정량이 얼마나 되나?" 둘을 동시에 본다 — 이걸 수식화한 것이 CR.

4.4 Setup avoidance — 같은 setup 끼리 묶기

- Implant 같은 area 는 wafer 종류가 바뀔 때 **setup 시간이 매우 길다 (60–80분)**.
- 도메인 규칙: 큐에 setup 같은 lot 이 있으면 그것을 우선 처리해 setup 전환 횟수를 줄인다 (`Setupavoidancerule`).
- Trade-off: 같은 setup 만 처리하면 다른 setup 의 lot 이 굶을 수 있음 → fairness 와 균형 필요.

4.5 Reservation — 병목 설비 미리 잡기

- 매우 비싼 / 부족한 설비 (예: Litho stepper) 는 빈자리가 났을 때 hot lot 이 먼 곳에서 도착할 때까지 *예약* 해두는 것이 fab 평균 cycle time 면에서 유리할 수 있다.
- 단순 FIFO 라면 hot lot 도착 전에 일반 lot 이 잡아버려 hot lot 이 굶음.

4.6 LTL dedication — Litho-to-lens dedication

- 한 lot 이 Litho 의 한 stepper 로 첫 layer 를 처리한 뒤, 후속 layer 를 **반드시 같은 stepper** 에서 처리해야 하는 제약 (lens-to-lens variation 회피).
- 데이터셋에서 73 step 이 LTL 제약을 가짐.
- Dispatch 시점에 이 제약을 만족하는 stepper 가 다른 lot 으로 막혀 있으면 lot 은 기다려야 한다.

4.7 CQT 회피 — Critical Queue Time

- "이 step 끝난 뒤 N분 안에 다음 step 에 도착해야 한다" (산화막 등 시간 민감 공정).
- N 을 넘기면 wafer 가 buf 라거나 손상 → scrap 또는 rework.
- 도메인 규칙: CQT-active lot 은 **CQT 만료 임박 정도로 추가 부스트** 받는다.
- 데이터셋의 CQT step: 264개.

4.8 도메인 규칙 → 우리 시뮬레이터의 dispatch rule 매핑

`Toolgroups.csv` 가 제시하는 도메인 규칙은 4가지 정도로 압축된다 (`SuperHotLotFIRST_and_Reservation`, `Setupavoidancerule`, ranking columns: `Highest Lotpriority` / `Least Setuptime` / `Critical Ratio` / `wake up Least Setuptime`). 본 시뮬레이터는 이를 더 일반화해 **14개 dispatch rule** 을 등록해 두고 비교한다 (`src/simulator/policies.py`):

- **Time-based**: `fifo`, `lifo`

- **Process-time-based**: `spt` (shortest processing time), `lpt`, `srpt` (shortest remaining), `lrpt`
- **Due-date-based**: `edd` (earliest due date), `least_slack`, `slack_per_remaining_step`, `critical_ratio`
- **Priority-aware (super-hot 우선)**: `priority_fifo`, `priority_spt`, `priority_edd`, `priority_least_slack`, `priority_cr_fifo`

발표 메시지: *"각 baseline rule 은 작업자의 1가지 직관을 코드화한 것이다. CR + 우선순위 + FIFO 의 조합인 `priority_cr_fifo` 는 도메인 best-practice 를 가장 가깝게 따라간 규칙."*

---

5. 베이스라인 성능

5.1 실험 세팅

- 데이터셋: `configs/default_simulation.json` 기본 (variable due dates 파일 사용)
- 정책 5종: `fifo`, `spt`, `edd`, `critical_ratio`, `priority_cr_fifo`
- 두 가지 scale 의 결과 보유:
  - **Full release** (167,129 lot release record 모두 시뮬레이션, until_minutes=129,500 분 ≈ 90일)
  - **1,000-lot smoke** (max_lots=1,000, until_date=2018-04-01)
- Seed: 42

5.2 결과 — Full release scale

`outputs/policy_comparison/overall_policy_summary.csv`:

Policy	Completed	Avg Cycle (min)	Avg Tardy (min)	Tardy lots	On-time %	CQT viol.
`fifo`	2,990	**43,014**	462.0	322	89.2	5,856
`spt`	2,992	43,481	567.1	400	86.6	4,350
`edd`	**3,181**	**39,361**	20.6	**19**	**99.4**	**2,815**
`critical_ratio`	2,921	45,482	**21.2**	255	91.3	5,750
`priority_cr_fifo`	2,926	44,178	24.3	164	94.4	5,505

읽는 법:

- 평균 cycle time 은 `edd` 가 가장 짧음 (39,361 분 ≈ 27.3 일).
- 평균 tardiness, max tardiness 모두 `edd`/`critical_ratio` 가 압도적으로 좋음. `fifo`/`spt` 는 매우 나쁨 (max tardy 28,000+ 분 = 19.5일 늦음).
- CQT violation 도 `edd` 가 최저.
- 완료 lot 수 (throughput) 는 큰 차이 없음 — 즉 **같은 throughput 안에서 어떤 lot 을 빨리 빼느냐의 차이가 정책의 본질**.

5.3 결과 — 1,000-lot smoke (more controlled)

`outputs/policy_comparison_1000_tool_physical/overall_policy_summary.csv`:

Policy	Completed	Avg Cycle (min)	Avg Tardy	Tardy	On-time %	CQT viol.	CQT viol. rate
`fifo`	815	36,574	46.81	10	98.77	543	2.45%
`spt`	781	36,122	34.95	9	98.85	413	1.95%
`edd`	793	**34,882**	21.82	4	99.50	**250**	**1.17%**
`critical_ratio`	799	36,122	**14.96**	**4**	**99.50**	489	2.27%
`priority_cr_fifo`	814	35,968	18.08	4	**99.51**	373	1.69%

Policy ranking (mean of 7 metrics, 낮을수록 좋음):

Rank	Policy	Mean rank
1	`priority_cr_fifo`	**1.71**
2	`critical_ratio`	1.86
3	`edd`	2.14
4	`spt`	3.86
5	`fifo`	5.00

5.4 정책별 강·약점 (발표용 해설)

Rule	강점	약점
FIFO	단순, fairness	납기·CQT·setup 모두 무시. cycle time / tardiness 최악
SPT	throughput	긴 lot 이 굶음. tardiness 나쁨
EDD	납기 압박 강하게 처리 → tardiness/CQT 우수	처리시간 무시 → 비효율
CR	tardiness 평균 최저	tail (max tardy) 큼, CQT 약간 약함
Priority + CR + FIFO	hot lot 보호 + 납기 + tie-breaking → 종합 1위	여전히 tool-local greedy

5.5 발표 메시지

> *"단순 FIFO 대비 도메인 지식을 살짝 넣은 priority+CR+FIFO 만으로도 평균 tardiness 가 462분 → 24분으로 줄어든다. 그러나 모든 baseline 은 결국 'tool-local greedy' 라는 공통의 한계를 가진다. 더 나아가려면 tool 의 시야 자체를 넓혀야 한다."*

---

6. 우리 접근 — DSM 으로 의사결정의 시야 넓히기

6.1 RL agent 도 (현재로서는) 일종의 베이스라인

- RL 은 reward 를 최대화하는 dispatch policy 를 학습한다.
- 그러나 흔한 RL 세팅에서 state 는 *그 tool 의* 큐 정보 위주이고, action 은 *그 tool 의* lot 선택. 즉 본질적으로 여전히 tool-local 의사결정이다.
- 그래서 **RL 자체는 우리 연구의 비교 기준선** 이지, 우리가 풀려는 문제의 답이 아니다.

6.2 Tool 단위 greedy 결정의 진짜 문제

상황 예시:

> **상황**: Litho_FE_92 가 idle. 큐에 lot A (Product 1, due 임박, CR=0.9), lot B (Product 5, due 여유, CR=2.1) 가 있다. CR 규칙은 A 를 고르라고 한다. **그런데**: A 의 *다음* 공정은 LithoMet_FE_19 인데, 그 toolgroup 큐가 이미 25개 lot 으로 막혀있다. A 를 보내봤자 거기서 또 기다린다. **반면**: B 의 다음 공정은 비어 있는 Diffusion 으로, 보내자마자 처리된다. **결과**: B 를 먼저 보내는 편이 fab 전체 throughput / cycle time 면에서 나을 수 있다.

이런 비교는 **tool 의 시야를 넘어서, downstream 까지 보아야** 가능하다. 모든 baseline rule 은 이걸 못 한다. RL 도 state 에 downstream 정보가 없다면 못 한다.

6.3 DSM (Design Structure Matrix) 의 역할

- DSM = 노드 = toolgroup, edge = 공정 흐름의 강도. 즉 **"어떤 toolgroup 다음에 어떤 toolgroup 이 자주 오는가"** 를 행렬로 표현.
- 본 연구는 SMT2020 의 10개 product 라우트를 종합해 toolgroup 간 **directed weighted graph** 를 만들고, 이를 binary DSM 으로 정제한 뒤 **클러스터링** 으로 *흐름이 강하게 결합된 toolgroup 모듈* 을 식별한다.
- 핵심 아이디어:

  > *"DSM 에서 같은 모듈로 묶이는 toolgroup 들은 서로 강하게 연결되어 있다. 한 곳의 결정이 다른 곳에 큰 파급을 준다. 의사결정 시 이 모듈 단위 정보를 함께 보면 더 global 한 결정이 가능하다."*

6.4 DSM 구축 파이프라인 (`src/dsm/`)

본 연구의 DSM 모듈은 다음 단계로 구성된다.

(a) Edge weight 계산

$$ w(\text{src} \to \text{dst}) = \sum_{\text{product, transition}} \text{product_weight} \cdot \text{type_factor} \cdot \text{decay}^{\text{lag}-1} $$

- `product_weight`: 모든 product 동일(`uniform`) 또는 수요 비례(`wspw`/`wafer_count`).
- `type_factor`:
  - sequence (라우트의 자연스러운 다음 step) — 1.0
  - **CQT** — 3.0 (강한 시간제약은 더 강하게 가중)
  - **rework** — `rework_probability / 100` (확률적으로 일어나는 사건이므로 expected 단위로 환산)
- `decay`: lag=2 이상의 step 까지 볼 때 멀어질수록 약하게 (default 0.35).
- `window`: 최대 lag (default 5).
- `exclude_same_area`: 같은 area 안의 자명한 연결 빼기 옵션.

(b) 정규화 + threshold → binary DSM

- min-max 정규화 후 `threshold` (default 0.05) 이상인 edge 만 1, 나머지 0.

(c) Directed-aware clustering

세 가지 방법 모두 directed 정보를 보존:

- **Hierarchical (Ward + cosine)**: 노드의 (in-pattern + out-pattern) feature → cosine distance → Ward linkage. 결정적, cluster 수 직접 지정.
- **MCL (Markov Clustering)**: column-stochastic transition matrix 위에서 random walk simulation → attractor 발견. directed flow 에 자연스러움.
- **Directed Louvain**: nx.DiGraph + Leicht-Newman directed modularity 최적화. cluster 수 자동.

(d) 시각화

- Block-diagonal 형태로 reorder + cluster 박스 그리기 → "어떤 toolgroup 들이 한 모듈인가" 가 한눈에 보임.

(e) 결과 산출 (CLI 1번 실행 → 자동 폴더)

하이퍼파라미터 조합으로 자동 생성된 폴더 안에:

- `dsm_network.csv` — 어디 → 어디 (rule 만들 때 직접 사용)
- `dsm_clusters.csv` — toolgroup → cluster id (어떤 게 묶이는지)
- `dsm_clusters.png` — 시각화
- `dsm_config_used.json` — 재현용 (k, modularity, edges 수 등 메트릭 포함)

6.5 DSM 기반 dispatch 의 가능성

DSM 결과로 할 수 있는 것:

1. **Downstream-aware 디스패치 feature**: 현 lot 이 가는 다음 toolgroup 의 cluster 부하를 lot priority score 에 반영.
2. **Cluster-level 우선순위**: 한 cluster 가 막혀있으면 그 cluster 로 가는 lot 의 우선순위를 낮춤 (혹은 높임 — 예약 의도).
3. **RL state augmentation**: RL agent 의 state 에 "cluster 부하 vector" 추가 → tool-local 시야 → cluster-level 시야 확장.
4. **CQT-aware**: CQT edge 가 cluster 안에 들어있으면, 그 cluster 안의 lot 은 CQT 우선 처리.
5. **Bottleneck 식별**: cluster 안에서 가장 강한 in-degree node = 그 cluster 의 bottleneck candidate.

6.6 차별화 포인트 (vs. 기존 연구)

측면	일반 RL / 도메인 rule	본 연구
의사결정 단위	tool 1대 / lot 1개	tool + DSM 모듈 (관련 있는 공정 묶음)
시야	한 큐 안의 lot	downstream cluster 부하까지
데이터 활용	현재 큐 상태	라우트 구조 + 현재 큐 상태
의사결정의 성격	greedy	(지향) global-aware
Baseline 가능성	정의 그대로 baseline	baseline 위에 얹는 augmentation

6.7 발표 한 줄 메시지

> *"Tool 한 대를 위한 결정에, fab 전역의 흐름 구조 (DSM) 를 한 가지 입력으로 더해주는 것. 우리는 dispatch 의 시야를 'tool-local' 에서 '공정 모듈-aware' 로 한 단계 끌어올린다."*

---

7. 발표 흐름 추천 (15–20분 발표 기준)

슬라이드 #	시간	내용	본문 절
1	0:30	Title + 한 줄 요약	0
2	1:00	반도체 팹 운영의 본질적 어려움	1
3	1:30	LV/HM fab 의 특수성 + 의사결정 단위의 한계	1.2–1.4
4	1:30	SMT2020 데이터셋 소개 + 규모	2.1–2.2
5	1:30	공정 area 와 라우트 구조	2.3–2.4
6	1:00	운영 제약들 (PM/CQT/Setup/Rework)	2.6
7	2:00	시뮬레이터 아키텍처와 검증 가능한 metric	3.1–3.6
8	1:30	작업자 도메인 dispatch 규칙	4
9	1:30	Baseline 5종 비교 결과 (표 + 그래프)	5.2–5.3
10	1:00	Tool-local greedy 의 한계 (예시 시나리오)	6.1–6.2
11	2:00	DSM 의 정의 + 구축 파이프라인	6.3–6.4
12	1:30	DSM 시각화 (cluster block-diagonal)	6.4 (e)
13	1:30	DSM 으로 dispatch 시야 확장하기 (계획)	6.5–6.6
14	1:00	정리: 메시지 3줄 + 향후 계획	0, 6.7

부록 A. 자주 받을 수 있는 질문 (FAQ)

Q1. 왜 DSM 인가? 그래프 임베딩 (GNN) 으로 직접 해도 되지 않나? A. 가능하다. 다만 (1) 라우트는 적은 수의 product 가 만드는 *해석 가능한* 구조라, GNN black-box 보다 DSM 형태의 명시적 구조가 도메인과 소통하기 쉽다. (2) DSM 결과를 RL state 에 feature 로 합치는 식으로 두 접근의 결합도 가능하다.

Q2. CQT 가중치 3.0, decay 0.35 같은 숫자는 어떻게 정했나? A. 모두 hyperparameter 다. `src/dsm/config.py` 의 dataclass default 가 첫 값이고, JSON config / CLI 로 모두 override 가능하다. 의미 있는 cluster modularity 가 나오는 범위에서 sensitivity test 가 따라야 한다.

Q3. LV/HM 이 아니라 HVLM (대량/소품종) 에서는 어떻게 다른가? A. SMT2020 dataset 1 이 HVLM. 제품 종류가 적어 setup 변경이 적고 batch / cascade 가 더 효과적이다. DSM 의 cluster 도 *area 와 거의 일치하는* 단순한 구조가 될 가능성 큼. 우리의 접근이 더 효과를 보는 곳은 **현재 LV/HM 같은 high-mix 환경**이다.

Q4. 시뮬레이터가 상용 SMT2020 과 정확히 같은가? A. 아니다. 핵심 객체와 제약 (PM, breakdown, setup, batch, cascade, CQT, rework, sampling, LTL) 은 모두 구현했지만, 일부 tool-level dispatch detail (예: stepper-specific 정렬, secondary tie-breaker) 은 단순화돼 있다. 그래서 절대값보다는 **정책 간 상대 비교** 에 의미가 있다.

Q5. RL 이 결국 같은 답을 학습할 수 있지 않나? A. 가능하다. 그러나 (1) state 안에 DSM-aware feature 를 명시적으로 넣어주면 학습 효율이 올라가고, (2) 그렇게 학습된 policy 는 도메인이 받아들이기 더 쉬운 형태의 설명을 가질 수 있다. 즉 DSM 은 RL 의 *대체* 가 아니라 *입력 공간의 향상* 이다.

부록 B. 추가로 슬라이드에 넣으면 좋을 그림 / 표

1. **데이터셋 통계 비교 다이어그램**: HVLM vs LVHM (제품 수, 라우트 길이, hot lot 비율)
2. **라우트 시각화**: Product 3 의 583 step 을 area 색상으로 색칠한 막대 (re-entrant 흐름 시각화)
3. **Toolgroup capacity heatmap**: 12 area × tool 수 (불균등성 강조)
4. **Tardiness distribution 박스플롯**: 5개 정책 비교 (`outputs/policy_comparison_1000_tool_physical/lots_*.csv` 사용)
5. **DSM clustered 그림**: `outputs/dsm_analysis/.../dsm_clusters.png` (이미 생성됨)
6. **Cluster x Area crosstab**: 각 cluster 가 어떤 area 들로 구성되는지

부록 C. 인용/참고 문헌

1. Kopp, Hassoun, Kalir, Mönch (2020). "SMT2020 — A Semiconductor Manufacturing Testbed". *IEEE Trans. Semiconductor Manufacturing*, 33(4), 522–531.
2. Mönch, Fowler, Mason (2013). *Production Planning and Control for Semiconductor Wafer Fabrication Facilities*. Springer.
3. Steward (1981). "The Design Structure System: A Method for Managing the Design of Complex Systems". *IEEE Trans. Engineering Management*, 28(3), 71–74. (DSM 원전)
4. Browning (2001). "Applying the Design Structure Matrix to System Decomposition and Integration Problems". *IEEE Trans. Engineering Management*, 48(3), 292–306. (DSM 클러스터링 응용)
5. Van Dongen (2000). *Graph Clustering by Flow Simulation*. PhD thesis, University of Utrecht. (MCL 원전)
6. Blondel, Guillaume, Lambiotte, Lefebvre (2008). "Fast Unfolding of Communities in Large Networks". *Journal of Statistical Mechanics*. (Louvain)
7. Leicht, Newman (2008). "Community Structure in Directed Networks". *Physical Review Letters*, 100, 118703. (Directed modularity)